

AI Engineer Intern — Practical Take-Home Assignment

1. Eligibility Criteria (Strict Filter)

Before starting this assignment, please confirm you meet our eligibility requirements:

- **Target Audience:** Strictly for **3rd or 4th-year undergraduate students** or **fresh graduates (Classes of 2024, 2025, or 2026)** in Computer Science, IT, AI/Data Science, or closely related engineering streams.
 - **Experienced Candidates:** **Do not submit** this assignment if you already have full-time software engineering experience. This assignment is solely for entry-level intern evaluation.
-

2. Role Reality & Work Breakdown (Mandatory Read)

We believe in complete transparency. This internship is **not** a 100% pure research or coding role. It is a hybrid execution-and-building role:

- **40% Manual Operations:** You will spend dedicated daily time executing manual prospecting tasks: finding verified emails, handling LinkedIn accounts, prospect discovery, verifying leads, and cleaning raw datasets.
- **60% AI Agent Engineering:** You will build autonomous web scrapers, browser agents, and LLM extraction pipelines designed specifically to **automate the exact manual workflows you execute**.

Mandatory Screening Question

In your submission email/form, you **must** include your explicit answer to this question:

"Are you 100% comfortable spending roughly 40% of your working hours on manual lead prospecting, email discovery, and account handling alongside your AI engineering tasks? (Yes / No)"

(Submissions answering 'No' or leaving this blank will be automatically disqualified.)

3. Technical Assignment: Autonomous Lead Enrichment Agent

Objective

Build a Python-based autonomous agent or pipeline that takes target company domains as input, crawls and parses their public web presence, and outputs structured intelligence using an LLM.

Input

Your script should accept an array/list of company domains. For testing, run your solution on these 3 targets:

1. postman.com
 2. supabase.com
 3. vapi.ai
-

Core Requirements

Step 1: Automated Browsing & Content Retrieval

- Fetch the homepage and discover relevant subpages (e.g., [/about](#), [/team](#), [/company](#), [/contact](#), [/pricing](#)).
- Handle dynamic, JavaScript-rendered content cleanly using headless browser automation (e.g., **Playwright**, **Puppeteer**, or **Selenium**).

Step 2: Context Pre-Processing & Token Optimization

- **Do not feed entire raw HTML trees into an LLM.**
- Extract clean markdown, text nodes, or specific DOM elements (strip out CSS, SVGs, scripts, and navigation boilerplate) to save LLM tokens and reduce latency.

Step 3: LLM Extraction (Structured Outputs)

- Use an LLM of your choice (OpenAI, Anthropic, or open-source models via Groq, Together AI, or Ollama).
- Use strict structured outputs (e.g., **Pydantic**, Instructor, or JSON Schema tool calling) to extract:
 - **Company Overview:** A concise 2-sentence summary of what they do.
 - **Target Audience / ICP:** Who their product is built for (e.g., "Developers building backend applications").
 - **Contact Points:** Any generic or public emails found on the site (e.g., [contact@](#), [sales@](#), [support@](#)).
 - **Key Leadership / Team Members:** Names, roles/titles, and LinkedIn profile URLs (if discoverable from the page content).

- **Data Confidence Score:** An estimated score between `0.0` and `1.0` indicating the quality/completeness of the extracted data.

Step 4: Fallback & Resilience

- Handle common web issues gracefully: 404 pages, bot blockers, timeouts, and missing elements. The script must never crash midway when one site fails.
-

4. Bonus Points (Optional, but gives you an edge)

- **Google/Search Integration:** Having the agent use a search engine (like SerpAPI, Tavily, or custom browser search) to look up external LinkedIn URLs for founders if not found on the direct website.
 - **Agentic Frameworks:** Implementing dynamic navigation using tools like **Browser-Use**, **LangGraph**, or custom multi-step tool-calling loops.
 - **Cost Tracking:** Logging the number of tokens used and the estimated API cost per scraped domain.
-

5. Submission Deliverables

Your submission must include:

1. **GitHub Repository Link:**
 - Clean, modular Python code.
 - A `requirements.txt` or `pyproject.toml` file.
 - A clear `README.md` explaining how to set up environment variables (`.env`) and run the project locally.
 2. **Sample Output File:**
 - A sample `output.json` or `output.csv` generated by running your script against the 3 test domains provided above.
 3. **Loom / Screen Recording (2 to 3 minutes max):**
 - A brief video walkthrough showing your code structure, running the script in the terminal, and showing the final extracted output.
 4. **Answer to the Operations Question:**
 - Your explicit confirmation regarding the 40% manual operations expectation.
-

6. Evaluation Rubric

Criterion	Weight	What We Are Looking For
Agent & Scraping Architecture	30%	Efficient DOM handling, headless browser integration, handling pagination/links without breaking.
LLM & Structured Output Quality	25%	Reliable Pydantic/JSON schemas, prompt robustness, token filtering (no raw HTML dumps).
Error Handling & Resilience	20%	Handling timeouts, anti-bot warnings, missing fields, and rate limits gracefully.
Code Quality & Documentation	15%	Clean functions, separation of concerns, type hints, and easy-to-follow README.
Loom Walkthrough	10%	Clear communication, conciseness, and demonstration of a working system.

7. Timeline & Submission Details

- **Deadline:** 48 hours to 72 hours from the receipt of this document.
- **Where to Submit:** Send an email to support@softwarebrio.com with the subject line:
[AI Intern Submission] - [Your Full Name]
- **Please include your linkedin profile in it.**